Project - High Level Design

on

Construction Planning Assistant Agent

Course Name: Agentic-Ai

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Vivek Sharma	EN22ME304116
2.	Jayesh Chouhan	EN23MEEL4001
3.	Sakina Modi	EN22EL301044
4.	Vinay Birla	EN22CS3011083
5.	Dev Singh Ghuraiya	EN22ME304022

Group Name:05D10

Project Number:DO-18

Industry Mentor Name:

University Mentor Name: Maya yadav

Academic Year:2025-26

Table of Contents

1. Introduction.
   - 1.1. Scope of the document.
   - 1.2. Intended Audience
   - 1.3. System overview.
2. System Design.
   - 2.1. Application Design
   - 2.2. Process Flow.
   - 2.3. Information Flow.
   - 2.4. Components Design
   - 2.5. Key Design Considerations
   - 2.6. API Catalogue.
3. Data Design.
   - 3.1. Data Model
   - 3.2. Data Access Mechanism
   - 3.3. Data Retention Policies
   - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
   - 7.1. Security Aspects
   - 7.2. Performance Aspects
8. References

1. Introduction

1.1 Scope of the Document

This High-Level Design document describes the architecture, system design, data model, and technical decisions for the Construction Planning Assistant Agent — an AI-powered agentic system that generates detailed construction project execution plans from natural language goals.

The document covers:

- Overall system architecture and layered design
- Agent execution flow and tool-calling mechanism
- Frontend interface design and component breakdown
- Data storage schema and persistence strategy
- API surface and integration points
- Non-functional requirements including security and performance

This document does not cover low-level implementation details, database query optimization, or deployment infrastructure configuration.

1.2 Intended Audience

Audience	Purpose
Software Developers	Understanding system architecture for development and extension
Technical Reviewers	Evaluating design decisions and architecture quality
Academic Evaluators	Assessing the agentic AI system design for project review
Future Maintainers	Onboarding to the codebase and understanding component responsibilities

1.3 System Overview

The Construction Planning Assistant Agent is a full-stack AI agent application that accepts a natural language construction goal from the user and autonomously generates a comprehensive, phase-by-phase execution plan.

Core Capabilities:

- Accepts free-form construction goals (e.g., "Plan a 3-story residential building")
- Decomposes the goal into 4 standard construction phases

- Autonomously calls 4 specialized tool functions per phase (materials, labor, permits, duration)
- Streams live agent reasoning and tool execution to the browser via Server-Sent Events
- Renders a structured result dashboard with metrics, phase table, Gantt chart, and agent log
- Persists all plans to a local JSON history file with full phase data

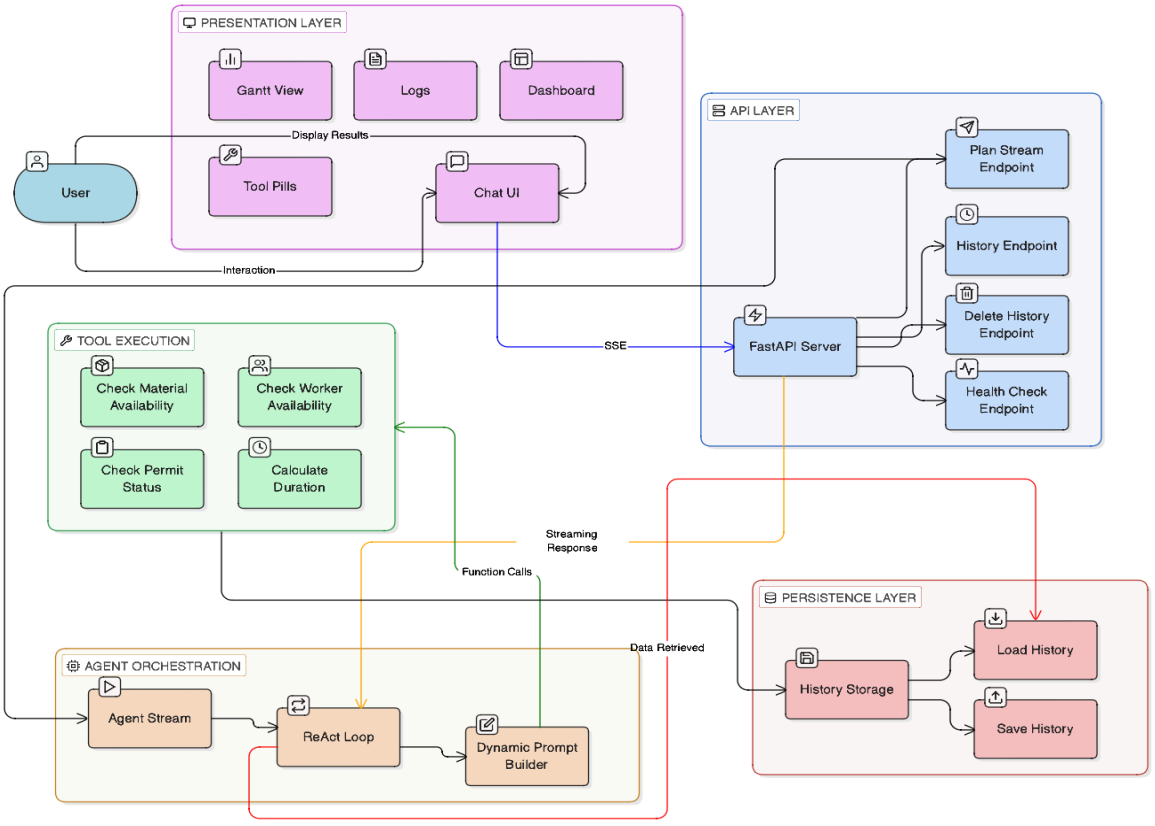
Technology Stack:

1. Layer: Technology
2. LLM: Llama 3.1 8B Instant via Groq API
3. Backend: Python 3.11 · FastAPI · Uvicorn
4. Agent Framework: Custom ReAct loop (no external agent library)
5. Frontend: Vanilla HTML · CSS · JavaScript (no framework)
6. Streaming: Server-Sent Events (SSE)
7. Persistence: JSON flat file (history.json)
8. Data Validation: Pydantic v2

2. System Design

2.1 Application Design

The system follows an Agent-Oriented Layered Architecture:

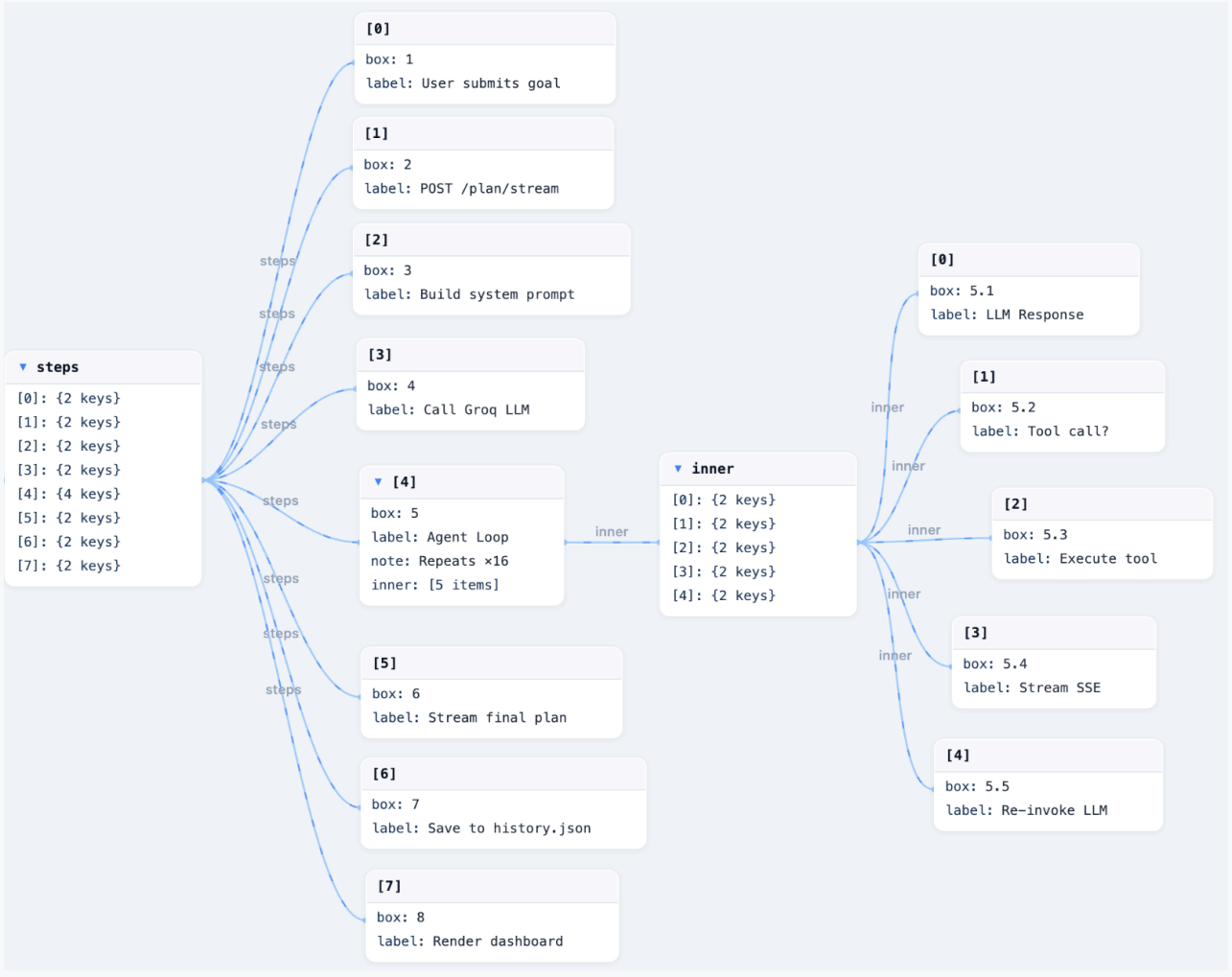


Key Module — agent_stream() — AI Agent Core Loop

The central function responsible for:

- Building a dynamic system prompt based on user-selected tool pills
- Sending messages to the Groq LLM (Llama 3.1 8B Instant)
- Interpreting finish_reason: tool_calls responses and executing tools
- Feeding tool results back into the conversation context
- Streaming SSE events to the browser for each tool call and the final response
- Saving the structured result to history.json on completion

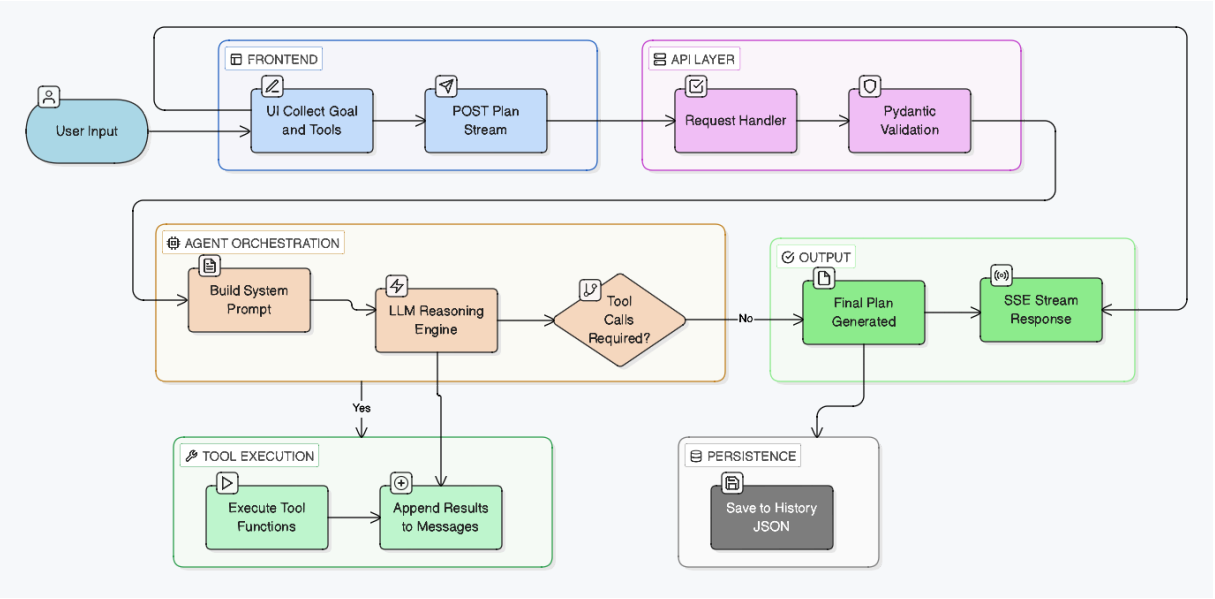
2.2 Process Flow (Agent Execution Flow)



Execution Steps:

1. User submits a construction goal and selects tool pills
2. Request sent to POST /plan/stream
3. build_system_prompt() constructs a context-aware prompt based on active pills
4. LLM begins reasoning over the task
5. Agent loop begins — LLM decides: continue reasoning OR trigger tool calls
6. If tool calls generated (up to 16 iterations — 4 phases × 4 tools):
    - Execute tool function
    - Stream {type: "tool_call"} SSE event to browser
    - Append result to messages[] context
    - Re-invoke LLM with updated state
7. Loop continues until finish_reason: stop or finish_reason: length
8. Final plan streamed as {type: "final"} SSE event
9. Plan saved to history.json
10. Browser populates dashboard tabs (Overview, Phases, Schedule, Agent Log)

2.3 Information Flow (Agent Data Flow)



SSE Event Types:

Event	When	Payload Fields
tool_call	Each tool execution	name, arg, result
final	LLM produces complete response	content, id
error	Rate limit or exception	message

2.4 Components Design

1. FastAPI Server (main.py)

- Exposes all API endpoints
- Serves the frontend SPA via StaticFiles
- Handles SSE streaming with StreamingResponse
- Manages request lifecycle, history read/write
- Contains build_system_prompt(), get_active_tool_defs(), agent_stream(), _build_entry()

2. AI Agent Loop (agent_stream)

Implements the ReAct (Reason + Act) pattern:

- Reason — LLM call via Groq API
- Act — Tool invocation based on LLM decision
- Observe — Tool result integration into messages[]
- Iterate — Loop up to 16 times before final response
- Resilient — Retries on tool_use_failed, handles rate limits, handles token truncation

3. Tool Layer (tools/construction_tools.py)

Four simulated tool functions representing external data sources:

1. **check_material_availability(phase)**: Simulates material stock system. Returns Material name + status.
2. **check_worker_availability(phase)**: Simulates labor registry. Returns Worker count + sufficiency.
3. **check_permit_status(phase)**: Simulates government permit database. Returns Permit type + approval state.
4. **calculate_duration(phase)**: Simulates project scheduling system. Returns Duration in days.

Key UI components:

- Home screen — rotating title, input card, tool pills
- Chat screen — user bubble, frequency waveform loading animation, analysis dropdown
- Result dashboard — 4 metric cards + 4 tabs (Overview, Phases, Schedule, Agent Log)
- Right log panel — 260px fixed panel with live THINK/CALL/OBS/DONE entries
- Left sidebar — history list with click-to-load and delete

4. Persistence Layer

- Flat JSON file: static/history.json
- Read: load_history() — returns list, handles missing file
- Write: save_history() — overwrites file with updated list
- Cap: 100 entries maximum (oldest dropped automatically)
- Served as a static file by FastAPI (accessible at /static/history.json)

2.5 Key Design Considerations

1. Scalable & Performant

- Stateless design with dynamic context (`messages[]`)
- Fast model (*llama-3.1-8b-instant*) + reduced tool calls (24 → 16) + SSE streaming

2. Reliable System

- Handles partial outputs, retries failures, and manages rate limits
- All errors streamed safely via SSE

3. Extensible & Maintainable

- Plug-and-play tools + UI-controlled activation (pills)
- Clean separation (LLM · Tools · State · Storage) with simple JSON persistence

2.6 API Catalogue

1. GET /

Serves frontend (index.html)
Response: text/html

2. POST /plan/stream

Request:

```
{
  "goal": "Plan a 3-story residential building",
  "tools": ["materials", "labor", "permits", "schedule"]
}
```

Stream (SSE):

```
data: {"type":"tool_call","name":"check_material_availability","result":"Concrete Delivered"}
data: {"type":"final","content":"## Project Overview..."}
data: {"type":"error","message":"Rate limit..."}
```

3. GET /history

Returns stored plans
Response: application/json

4. DELETE /history/{id}

Deletes a plan
Response: { "ok": true }

5. GET /health

System status
Response: { "status": "active" }

3. Data Design

3.1 Data Model

All plan data is stored in static/history.json as a JSON array. Each entry follows this schema:

```
{
  "id": "26832b1c-2ab7-4766-a042-4689426ce766",
  "project": {
    "goal": "Plan a 3-story residential building",
    "created_on": "2026-04-11",
    "created_at": "09:26 UTC",
    "total_duration_days": 55,
    "tools_enabled": ["materials", "labor", "permits", "schedule"]
  },
  "phases": {
    "Site Preparation": {
      "material": "Steel — Delayed",
      "labor": "6 workers (Sufficient)",
      "permit": "Structural permit — Pending Review",
      "duration": "7 days",
      "timeline": {
        "start_day": 1,
        "end_day": 7
      }
    },
    "Foundation": { "...": "..." },
    "Framing":    { "...": "..." },
    "Interior Finish": { "...": "..." }
  },
  "plan_markdown": "## Project Overview\n..."
}
```

3.2 Data Access Mechanism

Data access is handled by two synchronous utility functions in main.py:

```
def load_history() → list:
    # Reads history.json, returns [] if file missing or corrupt

def save_history(history: list):
    # Overwrites history.json with updated list (indent=2 for readability)
```

- Read path: GET /history → load_history() → return JSON array
- Write path: agent_stream() completes → _build_entry() → load_history() → insert at index 0 → save_history()
- Delete path: DELETE /history/{id} → load_history() → filter by id → save_history()
- No locking: Single-user local application — concurrent write safety is not required

---

3.3 Data Retention Policies

Policy	Detail
Maximum entries	100 plans (enforced at write time: `history[:100]`)
Eviction strategy	Oldest entries dropped (list is newest-first, slice from index 100)
Deletion	Manual via `DELETE /history/{id}` API or direct JSON edit
Backup	No automated backup — file is in `static/` and served as a static asset
Sensitive data	No PII stored — only construction goals and tool results

---

3.4 Data Migration

The history file has evolved through several schema versions during development. The current schema (v3) uses:

- project object (vs earlier meta object)
- phases as a dict with timeline sub-object
- plan_markdown key (vs earlier plan key)

Backward compatibility: loadHistoryItem() in app.js uses optional chaining with fallbacks:

```
const goal = item.project?.goal || item.meta?.goal || item.goal || 'Untitled';
const plan = item.plan_markdown || item.plan || '';
```

4. Interfaces

4.1 User Interface

The frontend is a single-page application served at http://localhost:8080.

Home Screen:

- Rotating title (5 phrases, 3.5s interval, CSS transition)
- Input card with auto-resizing textarea
- Tool pills (Materials, Labor, Permits, Schedule, Budget, Risk) — toggleable
- Model badge showing current LLM

Result Screen:

- Topbar with status dot, plan title, "New plan" button
- Thin progress bar (fills as tool calls complete)
- User message bubble (right-aligned)
- Frequency waveform loading animation (canvas-based, dual sine waves)
- Analysis dropdown (7 checkpoints, collapsible)
- 4 metric cards: Phases · Duration · Tool Calls · Readiness %
- 4 tabs: Overview · Phases · Schedule · Agent Log
- Right log panel (260px, live THINK/CALL/OBS/DONE entries)
- Left sidebar: history list with click-to-load and delete

4.2 API Interface

RESTful HTTP API with SSE streaming. See Section 2.6 for full catalogue.

4.3 LLM Interface

The system communicates with Groq's OpenAI-compatible API:

- Endpoint: https://api.groq.com/openai/v1/chat/completions
- Auth: Bearer token via GROQ_API_KEY environment variable
- Client: groq Python SDK (AsyncGroq)
- Tool calling: OpenAI function-calling format (tools, tool_choice: "auto")

5. State and Session Management

5.1 Request-Level State

The system is stateless at the HTTP level. Each POST /plan/stream request is fully independent.

Within a single request, the agent maintains a growing messages[ ] list:

```
messages = [
    {"role": "system",    "content": system_prompt},
    {"role": "user",      "content": goal},
    # After each LLM call:
    {"role": "assistant", "content": None, "tool_calls": [...]},
    # After each tool execution:
    {"role": "tool",      "tool_call_id": "...", "content": result},
    # ... repeats per tool call
    {"role": "assistant", "content": final_plan}  # final
]
```

This context window grows with each tool call iteration and is discarded when the request completes.

5.2 Client-Side State

The browser maintains application state in JavaScript variables:

Variable	Type	Purpose
historyData	array	Cached history from last /history fetch
activeHistoryId	string	Currently displayed plan ID
completedSteps	Set	Analysis steps marked complete
sidebarOpen	boolean	History sidebar open/closed state
waveAnimId	number	Canvas animation frame ID
titleIdx	number	Current rotating title index

No cookies, localStorage, or sessionStorage are used. State is lost on page refresh.

5.3 No Server-Side Sessions

There is no user authentication, session tokens, or server-side session storage. The application is designed as a single-user local tool.

6. Caching

6.1 Current Caching Strategy

The application has minimal caching appropriate for a local single-user tool:

Layer	Caching
LLM responses	None — each request generates fresh tool calls and plan
History data	In-memory in browser (`historyData` array), refreshed after each plan
Static assets	Browser default HTTP caching (no explicit cache headers set)
Tool results	None — tools are called fresh each request (intentional — simulates live data)

6.2 SSE Headers

The /plan/stream endpoint explicitly disables caching:

headers={ " Cache-Control ": " no-cache ", " X-Accel-Buffering ": " no " }

This ensures SSE events are not buffered by proxies or the browser.

6.3 Future Caching Opportunities

For production use, the following could be cached:

- Tool results per phase (Redis TTL cache — avoid redundant API calls)
- LLM responses for identical goals (semantic similarity cache)
- History data (browser localStorage for offline access)

7. Non-Functional Requirements

7.1 Security Aspects

- **API Key**: Stored in .env, loaded via python-dotenv, never exposed to frontend. Production Recommendation: Use secrets manager (AWS Secrets Manager, Vault).
- **Input validation**: Pydantic model validates request body type and structure. Production Recommendation: Add max length validation on goal field.
- **CORS**: Not configured (localhost only). Production Recommendation: Configure CORSMiddleware for production domains.
- **Authentication**: None — single-user local tool. Production Recommendation: Add API key or OAuth for multi-user deployment.
- **History file**: Served as static file — readable at /static/history.json. Production Recommendation: Move outside static/ directory for production.
- **Prompt injection**: System prompt uses f-string with user input. Production Recommendation: Sanitize user input before prompt construction.
- **Rate limiting**: Handled by Groq API (100k tokens/day free tier). Production Recommendation: Add application-level rate limiting for production.

7.2 Performance Aspects

- **Model**: Llama 3.1 8B Instant (~0.5–1s per tool call loop on Groq)
- **Tool calls per plan**: 16 (4 phases × 4 tools). Reduced from 24 for speed
- **Total plan generation time**: ~8–12 seconds. Depends on Groq API latency
- **Max output tokens**: 2048. Limits final response generation time
- **SSE perceived latency**: Near-zero. User sees tool calls streaming immediately
- **History file read**: Synchronous, O(n). Acceptable for ≤100 entries
- **Frontend bundle**: ~0 KB (no framework). Pure HTML/CSS/JS, no build step
- **Concurrent requests**: Not tested. FastAPI is async — supports concurrent SSE streams

Performance Optimizations Applied:

- Switched from llama-3.3-70b-versatile to llama-3.1-8b-instant — ~3× faster
- Reduced phases from 6 to 4 — 33% fewer tool calls
- Dynamic prompt construction — inactive pills remove tool definitions from API payload
- max_tokens: 2048 cap on final response

8. References

- FastAPI Documentation: https://fastapi.tiangolo.com
- Groq API Documentation: https://console.groq.com/docs
- Groq Model Deprecations: https://console.groq.com/docs/deprecations
- Server-Sent Events (SSE) — MDN: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
- Llama 3.1 8B Instant — Groq: https://console.groq.com/docs/models
- ReAct Paper: https://arxiv.org/abs/2210.03629
- OpenAI Function Calling: https://platform.openai.com/docs/guides/function-calling
- Pydantic v2 Documentation: https://docs.pydantic.dev/latest/